Supplemental Material

General Protein Pretraining or Domain-Specific Designs? Benchmarking Protein Modeling on Realistic Applications

D Abbreviation Statement

The Tab. 9 lists the abbreviations used throughout the paper, covering downstream applications, pretraining objectives, model architectures, datasets, and biological terminology.

Table 9: List of abbreviations used in this paper.

Abbreviation	Description
Downstream Applications	
PCS	Protein Cleavage Site prediction
PROTACs	Proteolysis-Targeting Chimeras
PLI	Protein–Ligand Interaction
PFA	Protein Function Annotation
MTP	Mutation Effect Prediction
Pretraining Objectives	
MLM	Masked Language Modeling
MVCL	Multi-View Contrastive Learning
PFP	Protein Family Prediction
Models & Architectures	
EGNN	Equivariant Graph Neural Network
SE(3) Trans	SE(3)-Equivariant Transformer
GVP	Geometric Vector Perceptron
D-Trans	Distance-aware Transformer
ProtBERT	ProteinBERT
ESM-C	ESM Cambrian
Datasets & Resources	
AFDB	AlphaFold Protein Structure Database
UR50	UniRef50 Protein Sequence Cluster
Biological terminology	
GO	Gene Ontology

E Reproducibility

Our implementations are based on widely adopted architectures, pretraining strategies, and downstream evaluation pipelines for protein modeling. The code is available at https://github.com/TrustApp-AI-Lab/protap. To ensure reproducibility, we provide detailed descriptions of the pretraining objectives in Section 3.2, along with the corresponding training configurations in Appendix I.2. Hyperparameters for all models, including learning rate schedules, batch sizes, and optimization settings, are explicitly reported in Section 5 and Appendix I.2. The datasets used for both pretraining and downstream evaluation are publicly available at https://huggingface.co/datasets/findshuo/Protap, and we specify their sources in Tab. 1 (b), and provide more details in Appendix H. The exact evaluation metrics are specified in Appendix H. These details, combined with the systematic comparison across architectures and strategies presented in Tab. 2 and Fig. 4, allow independent researchers to reproduce our results based on the manuscript alone.

F Supplemental Experiments and Observations

F.1 Descriptions and Categorization of Domain-Specific Models

The primary distinction between domain models and general models lies in their task-specific design. Domain models are tailored for particular tasks by enabling interaction between representations of different components and incorporating biochemical priors to enhance the encoder. In addition, domain models also leverage knowledge from other biochemical databases to preprocess the data, enabling the extraction of task-relevant key regions. Below, we summarize the characteristics of each domain model in relation to its respective tasks. Detailed task descriptions and model architectures are found in Appendix H and Appendix J.

Table 10: Domain Model Categorization

Category	Model
Biochemical Prior Enhancement	UniZyme, CLIPZyme, KDBNet, DeepFRI
Cross-Component Interaction	ET-PROTACs, DeepPROTACs
Hybrid	DPFunc, MONN

Enzyme-Catalyzed Protein Cleavage Site Prediction (PCS)

- **UniZyme**. UniZyme not only incorporates pretraining for enzyme active site prediction but also introduces energetic frustration, an intrinsic biophysical phenomenon that directly reflects active site properties and catalytic mechanisms. The integration of such biochemical priors enhances the generalization capability of the enzyme encoder.
- **ClipZyme**. ClipZyme formulates enzyme function prediction as a reaction-centric retrieval task, aligning enzyme representations with chemical reaction embeddings in a shared latent space. By explicitly modeling atom-mapped reaction graphs and constructing pseudo-transition states, the framework integrates reaction mechanism information that is specific to enzymatic catalysis.

Targeted Protein Degradation by Proteolysis-Targeting Chimeras (PROTACs)

- **DeepPROTACs**. DeepPROTACs circumvent explicit modeling of the ternary complex by encoding different components of the Target protein–PROTAC–E3 ligase system using separate neural network modules.
- **ET-PROTACs**. ET-PROTACs utilizes a cross-modal strategy and ternary attention mechanism, the model fully accounts for the cross-talk between PROTACs, target proteins, and E3 ligases, enabling more accurate modeling of ternary complex interactions.

Protein–Ligand Interactions (PLI)

- **KDBNet**. KDBNet does not model the entire protein, instead, it defines the binding pocket based on prior knowledge from the KLIFS database. In modeling kinases, it incorporates both geometric and evolutionary features, including backbone torsion angles and embeddings derived from the ESM language model.
- **MONN**. MONN is a multi-objective neural network designed to simultaneously predict non-covalent interactions and binding affinity between compounds and proteins, without relying on structural information.

Protein Function Annotation Prediction (PFA)

- **DeepFRI**. DeepFRI employs graph convolutional networks (GCNs) to process protein contact maps and integrates representations pretrained on Pfam sequences.
- **DPFunc**. DPFunc identifies key functional regions within protein structures and precisely predicts their associated biological functions by leveraging protein domain information.

F.2 Additional Observations

To provide a more comprehensive evaluation, we perform full parameter fine-tuning of the pretrained models on three downstream tasks, namely PCS, PROTACs, and PLI, with all encoder parameters updated during training. The corresponding results are presented in the Tab. 5. Generally, performing full-parameter fine-tuning on the entire model with a larger learning rate (1×10^{-4}) generally leads to slightly lower performance compared to the encoder-freezing setting. This suggests that overly aggressive full fine-tuning is less stable and often degrades ranking and regression metrics across specialized applications. This further reinforces our observation in RQ1: the effective use of pretrained models requires not only preserving pretrained knowledge but also providing the necessary representation adaptation capacity.

G Preliminaries

G.1 Protein Data

Proteins obtained through high-throughput sequencing are typically represented as sequences. However, using laboratory techniques such as X-ray crystallography, electron microscopy, XFEL, and nuclear magnetic resonance, the 3D structures of proteins can be elucidated. Unlike Computer Vision and Natural Language Processing, proteins can be characterized using various descriptions, such as 1D sequences, 2D graphs, and 3D structures.

Sequence. The most common representation is the amino acid sequence, typically in FASTA format using single-letter codes. Sequence-based models, originally RNNs and CNNs, are now dominated by Transformers due to their superior long-range modeling. Large pre-trained models such as ESM-2, ProteinBERT, and ProtTrans capture biochemical properties and evolutionary relationships, with structural information implicitly encoded in their learned representations.

2D Graph.. Proteins can also be described as molecular graphs, with residues as nodes and chemical or spatial relationships as edges. 2D graphs capture connectivity and topological features but lack explicit spatial details. Contact maps and models like GraphFormer encode residue-residue proximity or graph connectivity into the attention mechanism, partially reflecting structural information.

3D Structure. Many methods have been developed for modeling the 3D structures of proteins. 3D CNNs employ convolutional neural networks to extract local spatial features of proteins. Graph neural networks are widely used for representing protein and molecular structures, with examples including proteinMPNN, SE(3)-Transformer, GVP, and GearNet. GearNet applies contrastive learning for the pre-training of protein structures. Additionally, Transformer-M incorporates 3D information through distance matrices, enabling transformers to represent protein 3D structures while maintaining invariance. More recently, some works have begun to include protein surface information in modeling; for instance, ProteinINR integrates sequence, 3D structure, and surface information to enhance protein representations.

G.2 Terminology

This section provides detailed definitions of the professional terms used in the paper.

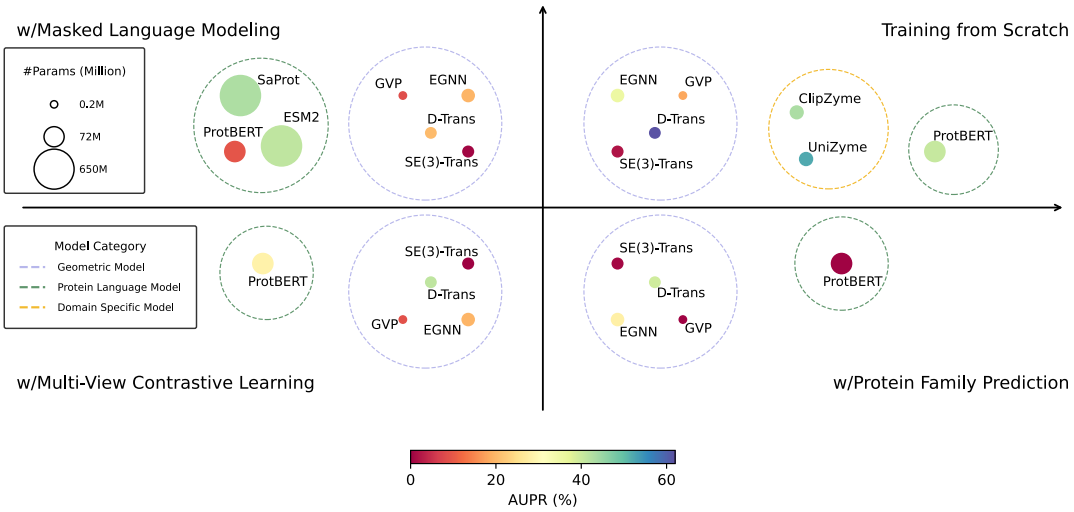
Protein Family. A protein family is a group of evolutionarily related proteins that typically share similar amino acid sequences, three-dimensional structures, and biological functions. Most members of a protein family are encoded by genes from a corresponding gene family, where each gene-protein pair has a one-to-one relationship. To date, more than 60,000 protein families have been identified, though exact numbers vary depending on the criteria and classification methods used.

Enzyme-catalyzed Protein Cleavage. Proteolytic enzymes first recognize short sequences or structural motifs within a substrate protein and then hydrolyze the peptide bond at the corresponding cleavage site, thereby fragmenting the polypeptide chain. This tightly regulated process governs protein maturation, turnover, and signaling, and its accurate in-silico prediction assists enzyme engineering and therapeutic target identification.

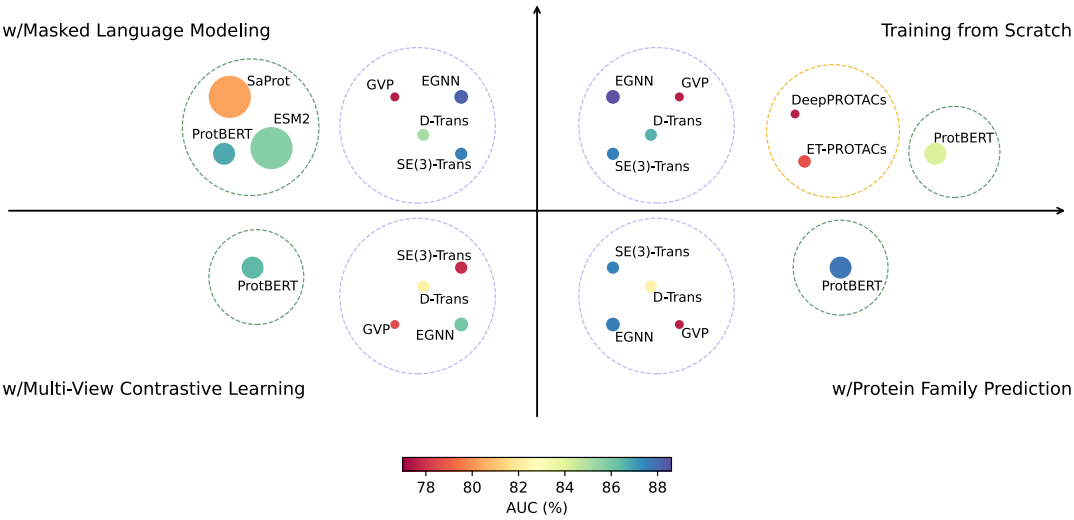
Proteolysis-Targeting Chimeras (PROTACs). A PROTAC is a heterobifunctional small molecule comprising (i) a warhead that binds the target (disease-relevant) protein, (ii) a chemical linker, and (iii) an E3-ligase-recruiting ligand. By simultaneously engaging the target protein and an E3 ubiquitin ligase, a PROTAC forms a ternary complex that drives poly-ubiquitination of the target, leading to its proteasomal degradation. Because PROTACs act catalytically and do not rely on classical active-site inhibition, they can eliminate previously "undruggable" proteins and often function at lower doses than conventional inhibitors.

Protein-Ligand Binding Affinity. Protein–ligand binding denotes the selective, high-affinity association between a protein and a small-molecule ligand to form a stable complex. Binding strength is quantified by the change in Gibbs free energy (ΔG) or related measures such as the dissociation constant (K_d); a more negative ΔG (or lower K_d) reflects stronger affinity. Reliable computational estimation of binding affinity accelerates virtual screening and lead optimization in drug discovery by prioritizing ligands most likely to bind their target proteins.

Gene Ontology (GO). Gene Ontology is a standardized vocabulary used to describe the functions of genes and proteins in a consistent and structured way. It provides a common language that allows researchers across different species and databases to describe what a gene product does, where it does it, and what biological processes it



(a) Comparisons between general and domain models on protein cleavage site prediction (PCS).



(b) Comparisons between general and domain models on targeted protein degradation by proteolysis-targeting chimeras (PROTACs).

Figure 7: Visualization of model size and performance across two specialized tasks: (a) PCS and (b) PROTACs. Each point represents a model, with size indicating parameter count and color intensity reflecting task performance. Each point represents a model, with its size corresponding to the number of parameters, ranging from 2M for GVP to 650M for ESM. Larger points indicate larger models. The color intensity of each point indicates performance, with darker shades representing stronger results. The dashed circular boundaries group models by category: models incorporating geometric information in color, protein language models in color, and domain-specific models in color.

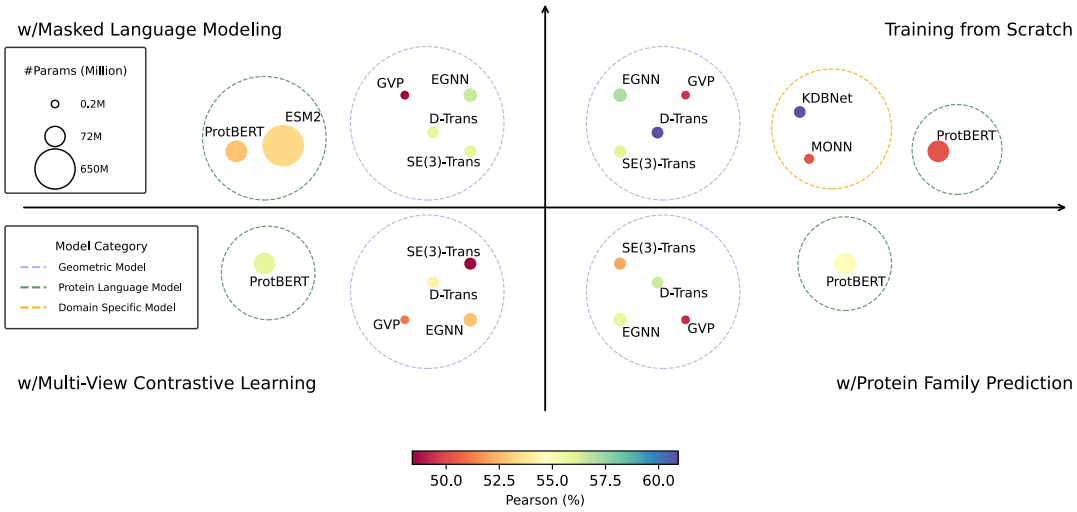
is involved in. GO is organized into three main categories, known as ontologies: (i) Biological Process (BP), which describes the broader biological goals that a gene or protein contributes to. (ii) Molecular Function (MF), which defines the specific biochemical activity of the gene product. (iii) Cellular Component (CC), which indicates the location within the cell where the gene product is active. Each GO term is assigned a unique identifier, e.g., GO: 0003677 for "DNA binding".

H Detail of Downstream Application and Dataset

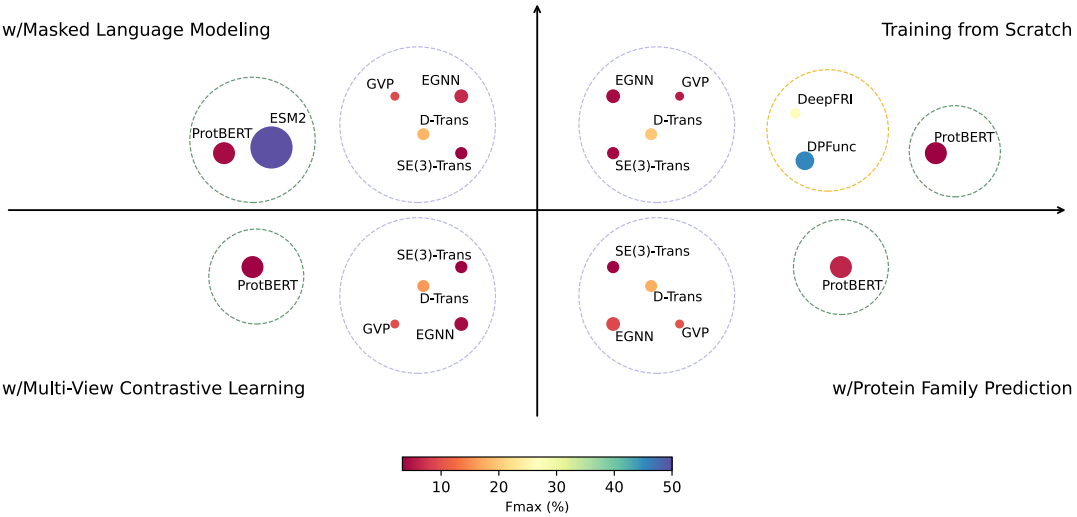
H.1 Enzyme-Catalyzed Protein Cleavage Site Prediction

Task Definition. Cleavage site prediction is formulated as a large-scale, imbalanced, residue-level binary classification problem. Let P_s denote a substrate protein of length $|R|$, and for each residue r_j ($j = 1, \ldots, |R|$), let $x_j \in \mathbb{R}^d$ be its feature vector. The objective is to learn a function

$$f : \mathbb{R}^d \longrightarrow \{0, 1\}$$



(a) Comparisons between general and domain models on Protein-Ligand Interaction (PLI).



(b) Comparisons between general and domain models on protein function prediction (PFA).

Figure 8: Visualization of model size and performance across two general tasks: (a) PLI and (b) PFA. Each point represents a model, with size indicating parameter count and color intensity reflecting task performance.

that outputs 1 if r_j is a cleavage site under catalysis by a given enzyme, and 0 otherwise. Formally, each substrate P_s is associated with a label vector $Y \in \{0,1\}^{|R|}$, where $Y_j = 1$ indicates cleavage at residue j. Training proceeds over residue-level examples $\{(x_j, Y_j)\}$ sampled from annotated cleavage datasets, enabling model generalization to varied substrate contexts.

Settings. For all pretrained models, we retain the same architectural hyperparameters as used during pretraining. During downstream training, we use a unified configuration across all models: the number of training epochs is set to 50, the learning rate to 1×10^{-4}, and the batch size to 24. A cosine annealing scheduler is employed to adjust the learning rate over time. To ensure robustness and reproducibility, we fix a set of five random seeds {42, 128, 256, 512, 1024}. For each model, we report the mean and standard deviation of its performance across these five runs. All experiments were conducted using 8 NVIDIA L40 GPUs. The average training time varied depending on the model size and task complexity, ranging from approximately 15 minutes to 5 hours.

Metric. The evaluation of enzyme-catalyzed protein cleavage site prediction is framed as a residue-level binary classification task under substantial class imbalance. Accordingly, the area under the receiver operating characteristic curve (AUC) and the area under the precision-recall curve (AUPR) are used as primary evaluation metrics. AUC quantifies the trade-off between true positive rate (TPR) and false positive rate (FPR) across various classification

thresholds, offering a global view of discriminative ability. However, due to the rarity of cleavage sites, AUPR is considered more informative, as it emphasizes the precision-recall behavior specific to the minority class. In particular, AUPR reflects the expected precision over all recall levels, which is critical for the reliable detection of enzymatic cleavage sites.

Dataset. The detailed information of the dataset is as follows:

- **Data Source**. The original data are from the MEROPS database. This dataset has a public website, available at the following address: http://cadd.zju.edu.cn/protacdb/
- **Pre-process**.
  - Prior studies have shown that slight sequence variations among enzymes within the same MEROPS category are negligible. As a result, we generalize the hydrolysis data from a specific substrate-enzyme interaction to all enzymes in that category. This allowed us to augment our dataset by linking each substrate not only to its originally annotated enzyme but also to other enzymes classified in the same MEROPS group. Finally, we selected two enzyme families, `C14.003` and `M10.003`, for evaluation.
  - We performed quality control on the raw data by filtering out entries with hydrolysis sites exceeding the maximum sequence length.
  - We converted the structural data of proteins in the dataset into a dictionary format and stored it in a `.pickle` file. The fields are as follows:

```
{
"Q5QJ38":{
    "name": "Q5QJ38",
    "seq": "MPQLLRNVLCVIETFHKYASEDSNGAT..",
   "coords": [[[-0.43, 25.50, -8.24], ..], ..],
    "cleave_site": [136]
    }
}
```

- **Data Format**. The sequence and structure of substrate proteins are stored in a `.pickle` file. The hydrolysis site information of substrate proteins is stored in a `.pickle` file as follows. It is worth noting that the hydrolysis site positions are 1-based indexed.

```
{
"P31001_MER0000622": [110, 263],
"O00232_MER0000622": [276, 334, 19],
...
}
```

- **Data Statistics**. The statistics of the dataset after preprocessing are summarized in Table 11. The training and test sets were split based on sequence similarity, ensuring that the sequence similarity between the test and training sets is below 60%.

Table 11: Statistics of the dataset after preprocessing.

Enzyme Family	#Train	#Test	Sites (Train)	Sites (Test)
C14.005	468	117	656	159
M10.003	375	92	953	157

- **Usage**. This dataset is used for a residue-level binary classification task. The goal is to predict the hydrolysis sites of a protein by a given enzyme family, based on its sequence or structure.
- **License**. We release a preprocessed version of the dataset under the MIT License. The original MEROPS database is provided under the terms of the GNU Library General Public License and is available at: https://www.ebi.ac.uk/merops/about/availability.shtml.

H.2 Targeted Protein Degradation by Proteolysis-Targeting Chimeras

Task Definition. This task is typically formulated as a binary classification problem. Given a PROTAC candidate and its corresponding POI and recruited E3 ligase, the goal is to predict whether the induced ternary complex will successfully trigger degradation. Formally, let the PROTAC molecule be decomposed into three modular components: warhead w, linker l, and E3-ligand el. Along with the protein of interest p and the E3 ligase eg, each component is processed through dedicated neural encoders:

$$\mathbf{h}_p = f_p(p), \quad \mathbf{h}_w = f_w(w), \quad \mathbf{h}_l = f_l(l), \quad \mathbf{h}_{el} = f_{el}(el), \quad \mathbf{h}_{eg} = f_{eg}(eg)$$

The resulting latent representations are concatenated to form a joint embedding that captures the structural and biochemical context of the ternary complex:

$$\mathbf{h}_{\text{concat}} = \text{concat}(\mathbf{h}_p, \mathbf{h}_w, \mathbf{h}_l, \mathbf{h}_{el}, \mathbf{h}_{eg})$$

This composite feature vector is passed through a multi-layer perceptron (MLP) with two fully connected layers to output the degradation prediction score:

$$\hat{y} = \sigma(\text{MLP}(\mathbf{h}_{\text{concat}})) \in [0, 1]$$

where $\hat{y}$ denotes the predicted probability of successful POI degradation. The model is trained using binary cross-entropy loss, with ground truth labels indicating degradation activity.

Settings. For this task, we retain the same architectural hyperparameter configuration as used during pretraining. To encode SMILES representations, we adopt the GVP architecture with a node dimensionality of 128, an edge dimensionality of 32, and a total of 3 layers. Models are trained for 50 epochs with a learning rate of 5e-4, a batch size of 24, and the Adam optimizer. We report the mean and standard deviation of performance across a fixed set of random seeds. All experiments are conducted on NVIDIA L40 GPUs, with per-run training time ranging from approximately 20 minutes to 6 hours, depending on the model and task complexity.

Metrics. The evaluation of PROTAC-induced protein degradation prediction is framed as a binary classification task. Accordingly, we adopt two widely used classification metrics: Accuracy measures the proportion of correct predictions among all samples, providing a straightforward indication of overall model performance. Area Under the Receiver Operating Characteristic Curve (AUC) quantifies the model's ability to distinguish degraders from non-degraders across varying decision thresholds.

Dataset. The detailed information of the dataset is as follows:

- **Data Source**. The original data was collected from the PROTAC-DB. This dataset has a public website, available at the following address: http://cadd.zju.edu.cn/protacdb/
- **Pre-process**.

- Following the processing strategy proposed by ET-PROTACs, we assign a degradation label to each triplet in the dataset. Specifically, we first examine whether a PROTAC entry contains DC_{50} measurements. If available, entries with DC_{50} values below 1000 nM are labeled as active, while those equal to or above 1000 nM are labeled as inactive. For entries lacking DC_{50} data, we instead inspect the reported degradation percentage of the target protein. If available, samples with degradation rates of at least 70% are considered active, and those below 70% are labeled as inactive. In cases where neither DC_{50} nor degradation percentage is reported, we fall back to IC_{50} values. Similarly, entries with IC_{50} below 1000 nM are labeled as active, and those above or equal to this threshold are labeled as inactive.
- For the target protein and E3 ligase, we retained only the entries with available structures in the AFDB. We filtered out the target proteins with the following UniProt IDs: `P36969`, `P03436`, `P0DTD1`, `Q12830`, and `G0R7E2`. The structural data of the target proteins and E3 ligases are obtained from AFDB. Then, we further convert the format into a dictionary and store it in a `.json` file. The fields are as follows:

```
[
"Q8IWV7":{
      "seq":"MADEEAGGTERMEISAELPQTPQRLASWW..",
       "coord":[[[21.99, 68.31,-49.90], ..],..]
    }
    ..
]
```

  For each structure, `coord` is a nested list of shape $(N \times 4 \times 3)$, representing the 3D coordinates of the backbone atoms N, Cα, C, and O for each residue in order, and N is the length of the amino acid sequence.
- For the warhead, linker, and E3-ligand, we use their SMILES strings to generate 20 conformers with RDKit's ETKDGv3 method. The conformers are optimized using a force field, and the lowest-energy conformer is selected. The final structure is saved in an SDF file.

- **Data Format**. The protein data, including the name, sequence, and coordinates, is stored in `.json` format. The structures of the warhead, linker, and E3-ligand are stored in SDF files. The label data is saved in a `.txt` file with the following format. The content below shows only key fields; for the complete list of fields, please refer to the full file.

```
uniprot  e3_ligase  linker    warhead     e3_ligand   label
Q00987    Q96SW2   linker_2  warhead_7   e3_ligand_7   1
Q00987    Q96SW2   linker_2  warhead_7   e3_ligand_16  1
P10275    P40337   linker_13 warhead_27  e3_ligand_27  0
P10275    P40337   linker_33 warhead_27  e3_ligand_27  0
```

- **Data Statistics**. The number of each component in the dataset is summarized in Table 12. The dataset was randomly split into a training set and a test set, with 80% used for training and 20% for testing.
- **Usage**. Based on the preprocessed reaction labels, this dataset is used for a binary classification task, where the goal is to predict the targeted degradation effect of PROTACs given the information of the target protein, E3 ligase, and the PROTACs.

Table 12: Statistics of PROTACs dataset components.

Target Protein	E3-Ligase	Warhead	Linker	E3-Ligand	Label: 1	Label: 0
154	10	552	1,534	131	2,244	1,878

- **License**. We release a preprocessed version of the dataset under the MIT License. Please refer to the usage license of the original data at: http://cadd.zju.edu.cn/protacdb/downloads, and follow the original authors' terms of use.

H.3 Protein–Ligand Interactions

Task Definition. Protein-ligand binding is the process by which proteins or various small molecules interact with high specificity and affinity to form a particular complex. A thorough understanding of the mechanisms underlying protein-ligand interactions is a prerequisite for gaining in-depth insights into protein function. The goal of rational drug design is to leverage structural data and knowledge of protein-ligand binding mechanisms to optimize the process of discovering new drugs. The driving force for protein-ligand binding arises from a combination of interactions and energy exchanges among proteins, ligands, water molecules, and buffering ions. Because the extent of protein-ligand binding is determined by the magnitude of negative ΔG, ΔG can be considered to determine the stability of any given protein-ligand complex, or equivalently, the binding affinity of a ligand for a given receptor. Experimentally measuring protein-ligand binding affinity is both time-consuming and complex, making it impractical to rely solely on experimental approaches for drug discovery from large compound libraries. In computational medicinal chemistry, predicting ligand binding affinity remains an open challenge. Existing deep learning methods attempt to directly predict binding affinity using affinity data from databases such as PDBBind.

From a machine learning perspective, protein-ligand affinity prediction is formulated as a regression problem, where the goal is to learn a function

$$f : \mathbb{R}^d \to \mathbb{R}$$

that maps the given input features X_i (e.g., protein structural or sequence data, ligand chemical descriptors, etc.) to a continuous value $y_i \in \mathbb{R}$, representing the quantitative binding affinity. Concretely, each protein-ligand pair (P_i, l_i) is associated with a feature vector $X_i \in \mathbb{R}^d$ and an affinity value y_i. The learning objective is to minimize the discrepancy between the predicted affinity $\hat{y}_i = f(X_i)$ and the experimentally measured (or otherwise ground-truth) value y_i, typically via metrics such as root mean squared error (RMSE) or mean absolute error (MAE). The function f can be learned using a training dataset

$$\{(X_i, y_i)\}_{i=1}^n,$$

and subsequently evaluated on unseen data to gauge its predictive performance and generalizability, ultimately utilized for drug screening.

Settings. For this task, we retain the same architectural hyperparameter configuration as used during pretraining. To encode SMILES representations, we adopt the GVP architecture with a node dimensionality of 128, an edge dimensionality of 32, and a total of 3 layers. The maximum length of the pocket amino acid

sequence is set to 85 residues, with shorter sequences padded and longer sequences truncated accordingly. Models are trained for 50 epochs with a learning rate of 5e-4, a batch size of 48 or 96, and the Adam optimizer. We report the mean and standard deviation of performance across a fixed set of random seeds. All experiments are conducted on NVIDIA L40 GPUs, with per-run training times ranging from approximately 20 minutes to 5 hours, depending on the model and task complexity.

Metrics. To assess a model's ability to predict binding affinity accurately, we adopt two complementary metrics: Mean Squared Error (MSE) and Pearson correlation coefficient. A lower MSE indicates more accurate regression of affinity values, while a higher Pearson coefficient reflects stronger linear correlation between predicted and true affinities. These metrics jointly capture both the precision and consistency of model predictions, providing a comprehensive evaluation of regression performance.

Dataset. The detailed information of the dataset is as follows:

- **Original Data Source.** The original data was collected from the KDBNet. The data is stored at https://www.dropbox.com/s/owc45bzbfn05ix4/data.tar.gz. Based on the existing DAVIS and KIBA datasets, the authors further extracted the binding pockets of proteins from protein-ligand complexes for use in modeling this task.
- **Preprocess.**
  - In the DAVIS portion of the KDBNet dataset, the protein `6FDY.U` contains missing coordinate values, which we filter out.
  - We convert the format of the protein structural data provided by the authors and store it as a dictionary in a JSON file:

```
[
"4WSQ.B":{
        "uniprot_id": "Q2M2I8",
      "seq": "EVLAEGGFAIVFLCALKRMVCKREIQIM..",
       "coord": [[[6.60,16.25,52.32], ..], ..]
    }
    ..
]
```

    For each structure, `coords` is a nested list of shape $(N \times 4 \times 3)$, representing the 3D coordinates of the backbone atoms N, Cα, C, and O for each residue in order, and N is the length of the amino acid sequence.
- **Format.** The protein data, including the name, UniProt ID, sequence, and coordinates, is stored in `.json` format. The label data is saved in a `.txt` file with the following format:

	drug	protein	Kd	y	protein_pdb
0	5291	AAK1	10000.0	5.0	4WSQ.B
1	5291	ABL1p	10000.0	5.0	3QRJ.B
2	5291	ABL2	10.0	7.99568	2XYN.C

  The ligand data is stored in SDF format.
- **Statistics.** As shown in Table 13, DAVIS contains 226 proteins, 64 compounds, and 14,464 interaction pairs, while KIBA includes 160 proteins, 1,986 compounds, and 89,958 pairs. The dataset was randomly split into a training set and a test set, with 80% used for training and 20% for testing. These datasets vary in scale and compound diversity, providing a comprehensive benchmark for model evaluation.

Table 13: Statistics of protein–ligand datasets.

Dataset	Protein	Ligand	Pair
DAVIS	226	64	14,464
KIBA	160	1,986	89,958

- **Usage.** This dataset is used for a regression task, where the goal is to predict the binding affinity for each protein-ligand pair.
- **License.** We release a preprocessed version of the dataset under the MIT License. The original dataset, also licensed under the MIT License, is available at: https://github.com/luoyunan/KDBNet/blob/main/LICENSE.

H.4 Protein Function Annotation Prediction

Task Definition. From a computational standpoint, protein function prediction is inherently a large-scale, sparse, and imbalanced multi-label classification problem. The goal is to predict multiple output labels from the given input features X_i. Let the set of labels be

$$\mathcal{L} = \{\text{BP}, \text{MF}, \text{CC}\}$$

Thus, each protein P_i is associated with a label vector $Y_i \in \{0,1\}^{|\mathcal{L}|}$, where $Y_i[j] = 1$ indicates that protein P_i is annotated with the j-th label, and $Y_i[j] = 0$ indicates it is not.

The goal is to learn a mapping function $f(X_i)$ to predict the label vector Y_i, i.e.:

$$f : \mathbb{R}^d \to \{0,1\}^{|\mathcal{L}|}$$

This function can be learned using the training dataset $\{(X_i, Y_i)\}_{i=1}^n$.

Settings. For this task, to rigorously assess the model's generalization ability, we included only those test sequences whose similarity to the training set was below 50%. we adopt the same architectural hyperparameter settings as used during pretraining to ensure consistency. Each model is trained for 50 epochs using the Adam optimizer with a learning rate of 5e-4 and a batch size of 24. To ensure robustness, we report the average performance and standard deviation across a predefined set of random seeds. All training is conducted on NVIDIA L40 GPUs, with individual runs taking between 2 minutes and 10 hours, depending on the model architecture and task complexity.

Metrics. Given the inherent sparsity of function annotation labels, where each protein is typically associated with only a small subset of possible functions, we evaluate model performance using Fmax and AUPR (Area Under the Precision-Recall Curve). These metrics are particularly suited for imbalanced multilabel classification tasks, where a higher Fmax and AUPR indicate better predictive capability in accurately identifying relevant functional annotations.

Dataset. The detailed information of the dataset is as follows:

- **Data Source**. The original data are from DeepFRI (https://github.com/flatironinstitute/DeepFRI), and the corresponding structural data are collected by ProteinWorkshop (https://zenodo.org/records/8282470/files/GeneOntology.tar.gz?download=1).
- **Pre-process**.

- We performed quality control on the raw data by filtering out entries with missing coordinates or with all function labels equal to 0.
- Based on the label annotations and protein information provided in the original dataset, we unified each protein entry into a dictionary format containing its name, sequence, coordinates, and functional labels. The final data was saved in a `.json` file, with each entry structured as follows:

```
[
  {
    "name": "2P1Z-A",
    "seq": "SKKAELAELVKELAVYVDLRRATLHARASR...",
    "coords": [[[6.43, 51.38, 15.44], ..], ..],
    "molecular_function": [0, 0, ..1,..],
    "biological_process": [0, 0, ..0,..],
    "cellular_component": [0, 0, ..1,..]
  },
  ...
]
```

  The fields `molecular_function`, `biological_process`, and `cellular_component` store one-hot encoded functional annotations. If you need Gene Ontology term IDs corresponding to the functional labels, please refer to the `label.tsv` file. The order of entries in this file strictly matches the position of each label in the one-hot vectors.

- **Data Format**. The protein name, sequence, coordinates, and functional labels are stored in a `.json` file. The corresponding Gene Ontology terms for the functional labels are provided in the `label.tsv` file.
- **Data Statistics**. Since *molecular function* defines the biological roles that a protein participates in, we restrict our functional prediction evaluation to *molecular function* only. The following Table 14 summarizes the label distribution across all proteins in the dataset. Label 1 refers to the total number of positive functional labels associated with proteins. Label 0 refers to the total number of functional labels not associated with the proteins.

Table 14: Statistics of positive and negative functional labels in the training and test sets.

Objective	#Train	#Test	Train		Test	
			Label: 1	Label: 0	Label: 1	Label: 0
Number	23,760	202	121,650	11,496,990	17,709	971,538

- **Usage**. This dataset is used for a multi-label classification task, where the goal is to predict the functional labels of each protein based on its sequence and structural information.
- **License**. We release a preprocessed version of the dataset under the MIT License. The original dataset is available under a BSD 3 license at https://github.com/flatironinstitute/DeepFRI/blob/master/LICENSE. And the protein structure data is released under the CC BY 4.0 license and is available at: https://openreview.net/pdf?id=sTYuRVrdK3

H.5 Mutation Effect Prediction for Protein Optimization

Task definition. The log-ratio between wild-type and mutant amino acid probabilities has been shown to be an effective estimator of mutational impact. In the zero-shot setting, we do not access any label information. Instead, we perform inference using a model pretrained with masked language modeling (MLM). The goal is to quantify the log-likelihood of protein variants under the background. The calculation is shown in the equation.

$$\sum_{t \in T} \log p(x_t = x_t^{mt} | S_{-t}) - \log p(x_t = x_t^{wt} | S_{-t})$$

where T is a set of positions where multiple mutations exist in the same sequence, and S is the wild-type sequence.

Settings. In the zero-shot setting, we do not use any label information, nor do we perform further training or fine-tuning. The prediction of mutation effects relies on the model's estimated probabilities for the mutant and wild-type amino acids at the mutation site. Therefore, pretraining methods whose models do not expose logits are unsuitable for this zero-shot setting, and no additional domain-specific models are incorporated in this study. Other types of pretraining methods are not suitable for this zero-shot scenario, and no additional domain-specific models are introduced in this application.

Metrics. Due to the non-linear relationship between protein function and fitness, the Pearson correlation coefficient is a suitable metric for evaluating model performance. Another evaluation metric is AUC, which assesses the model's ability to rank and discriminate between functionally neutral and deleterious mutations. During AUC calculation, `DMS_Score_Bin` is used as the ground-truth label (1 = positive, 0 = negative), while the model's continuous prediction scores serve as the decision function.

Dataset. The detailed information of the dataset is as follows:

- **Data Source**. The dataset we used to evaluate in this benchmark is from ProteinGym (https://proteingym.org/)
- **Pre-process**. Structural data is predicted by OmegaFold, where the input sequences for structure prediction are the wild-type sequences. We removed samples for which OmegaFold failed to generate structural predictions due to excessive sequence length.
- **Data Format**. The DMS substitution data provided by ProteinGym is formatted as shown below. The first column contains the mutation information, using 1-based indexing. For example, `F1I` indicates that the first amino acid in the wild-type sequence, originally phenylalanine (F), is mutated to isoleucine (I).

mutant	mutated_sequence	DMS_score	DMS_score_bin
F1I	ITLIELMIVIAIVGILA..	-3.598	0
F1L	LTLIELMIVIAIVGILA..	-0.678	0
F1V	VTLIELMIVIAIVGILA..	1.299	1
F1S	STLIELMIVIAIVGILA..	-0.127	0
T2A	FALIELMIVIAIVGILA..	0.786	1

- **Data Statistics**. After preprocessing, the dataset contains 201 proteins and 2,413,913 mutations. The detailed statistics of the preprocessed dataset are shown in Table 15.
- **Hosting**. The ProteinGym dataset is well-organized, and aside from adding structural information predicted by OmegaFold (while AlphaFold-predicted structures for DMS data are also

Table 15: Overall statistics of the ProteinGym DMS substitution dataset.

Objective	Proteins	Mutants
Number	201	2,413,913

available on the ProteinGym website), we did not perform any additional preprocessing. Therefore, we do not release a separate preprocessed version of the data.

- **Usage**. In the zero-shot setting, this evaluation involves no training process. Instead, it directly infers and quantifies the log-likelihood of protein variants under both sequence and structural context.
- **License**. The original dataset, ProteinGym, is available under a MIT license at https://github.com/OATML-Markslab/ProteinGym/blob/main/LICENSE.

I Pretrain Model Architecture and Experimental Setup

I.1 Model Architecture

This section details the models employed in this study.

Equivariant Graph Neural Networks (EGNN). EGNN is designed to process graphs where each node is associated with both feature embeddings and spatial coordinates. EGNN preserves equivariance under Euclidean transformations (translation and rotation) and node permutations, making it well-suited for modeling molecular and protein structures.

Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, each node $v_i \in \mathcal{V}$ has a feature vector $\mathbf{h}_i \in \mathbb{R}^{d_h}$ and a coordinate $x_i \in \mathbb{R}^n$. The Equivariant Graph Convolutional Layer (EGCL) updates node features and coordinates as follows:

$$\mathbf{m}_{ij} = \phi_e\left(\mathbf{h}_i^l, \mathbf{h}_j^l, \|\mathbf{x}_i - \mathbf{x}_j\|^2, a_{ij}\right), \tag{1}$$

$$\mathbf{x}_i^{l+1} = \mathbf{x}_i^l + C\sum_{j\neq i}(\mathbf{x}_i^l - \mathbf{x}_j^l)\cdot\phi_x(\mathbf{m}_{ij}), \tag{2}$$

$$\mathbf{m}_i = \sum_{j\neq i}\mathbf{m}_{ij}, \tag{3}$$

$$\mathbf{h}_i^{l+1} = \phi_h(\mathbf{h}_i^l, \mathbf{m}_i), \tag{4}$$

where ϕ_e is an edge function that computes a message embedding from node features, squared distance, and optional edge attributes a_{ij}. ϕ_x outputs a scalar weight for coordinate updates, based on m_{ij}. ϕ_h is a node update function. C is a normalization constant, typically $C = 1/(|\mathcal{V}| - 1)$.

Eq. (2) ensures that the coordinate update is equivariant to rotations and translations by acting as a learnable radial vector field. EGNN combines geometric awareness with standard message passing, making it a powerful architecture for modeling 3D protein structures.

SE(3) Transformer. The SE(3) Transformer is a neural architecture designed to model geometric data such as molecules and point clouds while respecting SE(3) symmetries, i.e., 3D rotations and translations. It achieves this through the integration of *Tensor Field Networks (TFNs)* for equivariant message passing and *invariant attention mechanisms* for weighted aggregation. This section presents a unified formulation of the SE(3) Transformer, encompassing both TFN convolution and attention-based update steps.

Formally, each node i in the graph is associated with a position $\mathbf{x}_i \in \mathbb{R}^3$ and a set of typed features $\mathbf{f}_i = \bigoplus_{\ell\geq 0}\mathbf{f}_i^\ell$, where $\mathbf{f}_i^\ell \in \mathbb{R}^{(2\ell+1)\times d}$ denotes a rank-ℓ feature tensor (type-ℓ representation of SO(3)) with d channels.

The output features at node i and type ℓ are computed as:

$$\mathbf{f}_{\text{out},i}^\ell = \mathbf{W}_{\text{self}}^{\ell\ell}\mathbf{f}_{\text{in},i}^\ell + \sum_{k\geq 0}\sum_{j\in\mathcal{N}_i\setminus\{i\}}\alpha_{ij}\,\mathbf{W}^{\ell k}(\mathbf{x}_j - \mathbf{x}_i)\,\mathbf{f}_{\text{in},j}^k. \tag{1}$$

The first term is a type-preserving self-interaction, and the second term aggregates messages from neighbors $j \in \mathcal{N}_i$ via a convolution-like operation using TFN kernels $\mathbf{W}^{\ell k}(\mathbf{x}_j - \mathbf{x}_i)$, modulated by scalar attention weights $\alpha_{ij} \in \mathbb{R}$.

The kernel $\mathbf{W}^{\ell k} : \mathbb{R}^3 \to \mathbb{R}^{(2\ell+1)\times(2k+1)}$ is constructed to ensure SE(3)-equivariance, and is defined as a linear combination of spherical harmonic projections:

$$\mathbf{W}^{\ell k}(\mathbf{x}) = \sum_{J=|\ell-k|}^{\ell+k}\varphi_J^{\ell k}(\|\mathbf{x}\|)\sum_{m=-J}^{J}Y_{Jm}(\widehat{\mathbf{x}})\,\mathbf{Q}_{Jm}^{\ell k}, \tag{2}$$

where $\widehat{\mathbf{x}} = \mathbf{x}/\|\mathbf{x}\|$, Y_{Jm} is the m-th spherical harmonic of order J, $\varphi_J^{\ell k}$ is a learnable radial function, and $\mathbf{Q}_{Jm}^{\ell k}$ are learnable matrices formed from Clebsch–Gordan coefficients. This construction guarantees equivariance under SE(3) transformations by disentangling radial and angular dependencies.

The attention weights α_{ij} are designed to be SE(3)-invariant and are computed via a dot-product attention mechanism:

$$\alpha_{ij} = \frac{\exp(\langle\mathbf{q}_i, \mathbf{k}_{ij}\rangle)}{\sum_{j'\in\mathcal{N}_i\setminus\{i\}}\exp(\langle\mathbf{q}_i, \mathbf{k}_{ij'}\rangle)}, \tag{3}$$

where the query vector $\mathbf{q}_i$ and key vector $\mathbf{k}_{ij}$ are both constructed from the input features through learned TFN mappings:

$$\mathbf{q}_i = \bigoplus_{\ell\geq 0}\sum_{k\geq 0}\mathbf{W}_Q^{\ell k}\mathbf{f}_{\text{in},i}^k, \quad \mathbf{k}_{ij} = \bigoplus_{\ell\geq 0}\sum_{k\geq 0}\mathbf{W}_K^{\ell k}(\mathbf{x}_j - \mathbf{x}_i)\,\mathbf{f}_{\text{in},j}^k. \tag{4}$$

Here, $\mathbf{W}_Q^{\ell k}$ and $\mathbf{W}_K^{\ell k}$ are TFN-type filters that output representations in the same basis, ensuring that the dot product is invariant under common SO(3) actions.

Equivariance is preserved in the message-passing path through the linear combination of TFN kernels. The attention mechanism maintains invariance because it operates entirely in scalar (dot-product) space between representations of the same type, which are invariant under group actions due to orthonormality properties of spherical harmonics.

GVP. The Geometric Vector Perceptron (GVP) is a neural module designed for learning over geometric data, in which each entity (e.g., an amino acid residue or atom) is represented by both scalar and vector features. Formally, given a pair $(\mathbf{s}, \mathbf{V})$ where $\mathbf{s} \in \mathbb{R}^n$ denotes scalar features and $\mathbf{V} \in \mathbb{R}^{\nu\times 3}$ denotes geometric vector features, the GVP outputs a new pair $(\mathbf{s}', \mathbf{V}') \in \mathbb{R}^m \times \mathbb{R}^{\mu\times 3}$. This transformation is designed to ensure that the scalar outputs remain invariant while the vector outputs are equivariant with respect to rotations and reflections in $\mathbb{R}^3$.

The GVP proceeds via the following algorithm:

Algorithm 1 Geometric Vector Perceptron (GVP)

Input: Scalar features $\mathbf{s} \in \mathbb{R}^n$, vector features $\mathbf{V} \in \mathbb{R}^{\nu \times 3}$
Output: Updated features $(\mathbf{s}', \mathbf{V}') \in \mathbb{R}^m \times \mathbb{R}^{\mu \times 3}$

$\mathbf{V}_h \leftarrow \mathbf{W}_h \mathbf{V}$
$\mathbf{V}_\mu \leftarrow \mathbf{W}_\mu \mathbf{V}_h$
$\mathbf{s}_h \leftarrow \|\mathbf{V}_h\|_2$ (row-wise)
$\mathbf{v}_\mu \leftarrow \|\mathbf{V}_\mu\|_2$ (row-wise)
$\mathbf{s}_{h+n} \leftarrow \texttt{concat}(\mathbf{s}_h, \mathbf{s})$
$\mathbf{s}_m \leftarrow \mathbf{W}_m \mathbf{s}_{h+n} + \mathbf{b}$
$\mathbf{s}' \leftarrow \sigma(\mathbf{s}_m)$
$\mathbf{V}' \leftarrow \sigma_+(\mathbf{v}_\mu) \odot \mathbf{V}_\mu$ **return** $(\mathbf{s}', \mathbf{V}')$

Here, $\mathbf{W}_h$, $\mathbf{W}_\mu$, and $\mathbf{W}_m$ are learnable linear transformations, while σ and σ_+ are nonlinear activation functions (e.g., ReLU, GELU, or their variants). The vector norm computations $\|\cdot\|_2$ are row-wise and used to extract invariant scalar information from the geometric vectors, which is then injected into the scalar pathway before transformation. The final output $\mathbf{V}'$ is modulated via element-wise multiplication with a positive gating function $\sigma_+(\mathbf{v}_\mu)$ to preserve equivariance.

The GVP-GNN updates node embeddings $h_v^{(i)}$ via message passing:

$$h_m^{(j\to i)} = \text{GVP}\left(\texttt{concat}(h_v^{(j)}, h_e^{(j\to i)})\right), \tag{1}$$

$$h_v^{(i)} \leftarrow \text{LayerNorm}\left(h_v^{(i)} + \frac{1}{k'}\sum_{j\in\mathcal{N}_i} \text{Dropout}(h_m^{(j\to i)})\right), \tag{2}$$

Here, GVP$(\cdot)$ denotes a sequence of three GVPs. The embeddings $h_v^{(i)}$ and $h_e^{(j\to i)}$ correspond to node i and edge $(j \to i)$, respectively, as previously defined. The message $h_m^{(j\to i)}$ is computed from these embeddings and represents the information passed from node j to node i. The variable k' denotes the number of incoming messages, which equals k unless the protein contains fewer than k amino acid residues. Then, a feed-forward point-wise layer is utilized to update the node embeddings at all nodes i:

$$h_v^{(i)} \leftarrow \text{LayerNorm}\left(h_v^{(i)} + \text{Dropout}(\text{GVP}(h_v^{(i)}))\right), \tag{3}$$

where GVP$(\cdot)$ is a sequence of two GVPs. This architecture allows for expressive, symmetry-aware modeling of protein geometry with built-in equivariance, while remaining efficient and conceptually simple.

ProteinBERT. ProteinBERT is a denoising autoencoder for proteins, inspired by BERT but with a distinct architecture. It takes two inputs: amino acid sequences and GO annotations. The architecture consists of parallel local and global pathways. Local representations are 3D tensors of shape $B \times L \times d_{\text{local}}$ (with $d_{\text{local}} = 128$), and global representations are 2D tensors of shape $B \times d_{\text{global}}$ (with $d_{\text{global}} = 512$).

Each input sequence is embedded into local features by a shared position-wise embedding layer, while annotations are mapped into global features via a fully connected layer. The model applies six transformer-like blocks, each updating local features using both narrow and dilated convolutions (kernel size 9, dilation 1 and 5), followed by feedforward layers. The global path consists of two fully connected layers per block.

Local-global interaction occurs via (1) a broadcast fully connected layer from global to local, and (2) a global attention layer from local to global. Global attention has linear complexity and uses trainable projection matrices W_q, W_k, and W_v (with $d_{\text{key}} = 64$, $d_{\text{value}} = 128$), and $n_{\text{heads}} = 4$ per block. All activations use GELU.

D-Transformer. D-Transformer augments the Transformer with 3D knowledge by adding pairwise Cα distances to the self-attention scores, similar to Transformer-M. For a protein of length L, let $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_L] \in \mathbb{R}^{L\times d}$ be residue embeddings and $D(i,j) = \|\mathbf{r}_i - \mathbf{r}_j\|_2$ the Euclidean distance between residues i and j.

Each distance is first expanded with a learnable Gaussian radial basis:

$$\phi_{\text{dist}}(D(i,j)) = \left[\exp\left(-\frac{(D(i,j)-\mu_k)^2}{2\sigma_k^2}\right)\right]_{k=1}^{K}, \tag{1}$$

where $\{\mu_k, \sigma_k\}_{k=1}^K$ are trainable parameters. A multilayer perceptron maps this K-vector to a scalar bias $b_{ij} = \text{MLP}(\phi_{\text{dist}}(D(i,j)))$.

For one attention head, the unnormalised score is

$$A_{ij} = \frac{(\mathbf{h}_i W_Q)(\mathbf{h}_j W_K)^\top}{\sqrt{d}} + b_{ij}, \tag{2}$$

with $\mathbf{h}_i$ the hidden state at residue i and learned $W_Q, W_K \in \mathbb{R}^{d\times d}$. Because b_{ij} depends only on distances, A_{ij} is invariant to global rotations, translations, and residue permutations. Normalised weights and value aggregation follow standard Transformer rules:

$$\alpha_{ij} = \text{softmax}_j(A_{ij}), \qquad \mathbf{z}_i = \sum_{j=1}^{L} \alpha_{ij}(\mathbf{h}_j W_V), \tag{3}$$

where $W_V \in \mathbb{R}^{d\times d}$. A position-wise feed-forward network with residual connections completes each layer, and stacking multiple such layers yields a lightweight architecture that leverages structural information without coordinate updates.

ESM2. ESM-2 is a family of transformer-based protein language models trained to predict masked amino acids in protein sequences. Compared to its predecessor ESM-1b, ESM-2 introduces architectural refinements, improved training procedures, and is scaled across model sizes ranging from 8M to 15B parameters. The model is trained using a standard masked language modeling (MLM) objective: for 15% randomly masked positions in a sequence, the model predicts the identity of each masked amino acid based on its unmasked context.

Formally, the training objective is:

$$\mathcal{L}_{\text{MLM}} = \sum_{i\in M} \log p(x_i \mid x_{\setminus M}), \tag{1}$$

where M is the set of masked positions and $x_{\setminus M}$ denotes the observed amino acids. Training is performed on ~65 million sequences sampled from ~43 million UniRef50 clusters, covering ~138 million UniRef90 entries.

Despite its unsupervised formulation, ESM-2 learns rich structural features purely from sequence data. The model achieves state-of-the-art performance on several protein structure prediction benchmarks and surpasses previous models, e.g., ESM-1b, ProteinBERT.

I.2 Pretrain Model Experimental Setup

In this section, we provide more details about the pretraining.

Table 16: Hyperparameter setting of the EGNN pre-training.

MLM	
Batch Size	48
Learning Rate	1e-3
Warmup Ratio	0.05
Mask Ratio	0.15
Hidden Dimension	512
Layer Depth	3
MVCL	
Batch Size	96
Learning Rate	1e-3
Warmup Ratio	0.05
Subsequence Length	50
Max Node	50
Temperature	0.0099
Hidden Dimension	512
Layer Depth	3
PFP	
Batch Size	48
Learning Rate	1e-3
Warmup Ratio	0.05
Temperature	0.01
Hidden Dimension	512
Layer Depth	3

Table 17: Hyperparameter setting of the SE(3) Transformer pre-training.

MLM	
Batch Size	48
Learning Rate	1e-2
Warmup Ratio	0.05
Mask Ratio	0.15
Hidden Dimension	36
Layer Depth	2
MVCL	
Batch Size	96
Learning Rate	1e-2
Warmup Ratio	0.05
Subsequence Length	50
Max Node	50
Temperature	0.0099
Hidden Dimension	36
Layer Depth	2
PFP	
Batch Size	48
Learning Rate	1e-3
Warmup Ratio	0.05
Temperature	0.0099
Hidden Dimension	36
Layer Number	2

Table 18: Hyperparameter setting of the D-Transformer pre-training.

MLM	
Batch Size	16
Learning Rate	1e-4
Warmup Ratio	0.1
Mask Ratio	0.15
Hidden Dimension	256
Layer Depth	6
MVCL	
Batch Size	16
Learning Rate	1e-4
Warmup Ratio	0.1
Subsequence Length	50
Max Node	50
Temperature	0.01
Hidden Dimension	256
Layer Depth	6
PFP	
Batch Size	16
Learning Rate	1e-4
Warmup Ratio	0.1
Temperature	0.01
Hidden Dimension	256
Layer Number	6

Table 19: Hyperparameter setting of the GVP pre-training.

MLM	
Batch Size	64
Learning Rate	1e-4
Warmup Ratio	0.1
Mask Ratio	0.15
Node Dimension	(155,16)
Layer Depth	3
MVCL	
Batch Size	64
Learning Rate	1e-4
Warmup Ratio	0.1
Subsequence Length	50
Max Node	50
Temperature	0.01
Node Dimension	(155,16)
Layer Depth	3
PFP	
Batch Size	64
Learning Rate	1e-4
Warmup Ratio	0.1
Temperature	0.01
Node Dimension	(155,16)
Layer Number	3

Table 20: Hyperparameter setting of the ProteinBERT pre-training.

MLM	
Batch Size	48
Learning Rate	1e-4
Warmup Ratio	0.05
Mask Ratio	0.15
Hidden Dimension	512
Layer Depth	12
MVCL	
Batch Size	96
Learning Rate	1e-4
Warmup Ratio	0.05
Subsequence Length	50
Max Node	50
Temperature	0.0099
Hidden Dimension	512
Layer Depth	12
PFP	
Batch Size	48
Learning Rate	1e-4
Warmup Ratio	0.05
Temperature	0.0099
Hidden Dimension	512
Layer Depth	12

Pre-training Data. The structural information used for pretraining is derived from the AFDB Swiss-Prot dataset, while the functional and family annotations are obtained from UniProt Swiss-Prot. Swiss-Prot is a high-quality, manually curated protein sequence database within UniProt. Its goal is to provide comprehensive and biologically meaningful annotations of protein sequences, such as their functions, families, and maintain minimal redundancy. The AFDB Swiss-Prot dataset is a subset of the AlphaFold Protein Structure Database that specifically contains structure predictions for all protein sequences corresponding to UniProt Swiss-Prot entries. Currently, the dataset includes a total of 542,378 entries. ESM-2 and SaProt are pretrained on UniRef sequences clustered at the 50% sequence identity level. ESM-C is also pretrained on UniRef but with clustering at the 70% sequence identity level, and further incorporates protein sequences from the MGnify and Joint Genome Institute (JGI) databases, resulting in 83M, 372M, and 2B clusters for UniRef, MGnify, and JGI, respectively.

Settings. We adopt task and architecture-specific training and model hyperparameters across different pretraining tasks and model types. Detailed configurations are summarized in Tab. 16-20.

For the Multi-View Contrastive Learning (MVCL) task, we set the maximum length of sub-sequences to 50. To extract substructures in 3D space, we randomly sample a central amino acid and collect all residues within a 15 Å Euclidean radius, with the subspace length capped at 50 residues. Temperature values used in the contrastive objective are adapted for each model architecture to ensure stable training dynamics.

For the Protein Family Prediction (PFP) task, we address the sparsity of family labels by adopting a negative sampling strategy. During training, family labels are embedded into the same representation space as proteins, and the model is optimized to align protein embeddings with their corresponding family embeddings. Simultaneously, embeddings of proteins from other families within the same batch are pushed apart, following a contrastive learning framework. A temperature coefficient is introduced to scale the similarity scores and enhance training stability.

J Domain-Specific Model Architecture

KDBNet. KDBNet is a graph neural network model designed for predicting kinase–small molecule binding affinity. It constructs heterogeneous graph pairs from the 3D structures of protein binding pockets and small molecules, and employs two structure-aware GNNs to learn their respective representations. For the protein, the model focuses on 85 binding site residues defined by the KLIFS database to build a structure graph $\mathcal{G}_p = (\mathcal{V}*p, \mathcal{E}*p)$, where nodes represent residues and edges are formed between C$*\alpha$ atoms within 8 Å. Each residue node includes three types of features: one-hot encoding of amino acid type, geometric features based on backbone conformation (e.g., $(\sin\phi_i, \sin\psi_i, \sin\omega_i, \cos\phi_i, \cos\psi_i, \cos\omega_i)$), and evolutionary embeddings obtained from the ESM language model. Edge features consist of four components: radial basis function (RBF) encoding of distances, local frame-based projections of relative direction vectors, rotational quaternions between residues, and relative position encodings using a Transformer-based function $E * \mathrm{pos}(c_j - c_i)$. The small molecule is represented as a graph $\mathcal{G}_d = (\mathcal{V}_d, \mathcal{E}_d)$, where atoms are nodes and edges connect atom pairs within 4.5 Å. Each atom node includes both 3D coordinates and a 66-dimensional scalar feature vector describing chemical properties, while edge features include unit directional vectors and RBF-encoded distances. For encoding protein structure, KDBNet uses a Graph Transformer, where each layer updates node features via the following attention mechanism:

$$h_i^{(\ell)} = W_1^{(\ell)} h_i^{(\ell-1)} + \sum_{j \in \mathcal{N}(i)} \alpha_{ij} \left(W_2^{(\ell)} h_j^{(\ell-1)} + W_3^{(\ell)} e_{ij} \right). \quad (1)$$

With attention weights defined as:

$$\alpha_{ij} = \mathrm{softmax}\left(\frac{\left(W_4^{(\ell)} h_i^{(\ell-1)} \right)^\top \left(W_5^{(\ell)} h_j^{(\ell-1)} + W_3^{(\ell)} e_{ij} \right)}{\sqrt{d_\ell}} \right). \quad (2)$$

Three such graph convolution layers are stacked with Leaky ReLU activations, and global sum pooling is applied to obtain the final protein embedding. For small molecules, KDBNet adopts the GVP-GNN architecture, which jointly models vector and scalar features to maintain rotational and translational equivariance. Each atom node is represented as a tuple (v_i^v, v_i^s) and each edge as (e_{ij}^v, e_{ij}^s), where the vector part allows direct alignment with atomic coordinates. The final protein and drug embeddings are passed through fully connected layers to regress the binding affinity.

DeepPROTACs. DeepPROTAC is a deep learning framework designed to predict the degradation efficacy of PROTAC molecules by integrating structural and chemical information from multiple components: the protein of interest (POI), the E3 ligase, and the

PROTAC compound itself. The input includes five parts: the POI binding pocket, the E3 ligase binding pocket, the warhead (the PROTAC moiety binding to the POI), the E3 ligand (binding to the E3 ligase), and the linker represented as a SMILES string. For the four molecular structures (POI pocket, E3 pocket, warhead, and E3 ligand), atom-level graphs are constructed and processed using Graph Convolutional Layers (GCLs) followed by max pooling to obtain fixed-size feature embeddings. The linker is embedded using an LSTM network to capture sequential chemical features. All five embeddings are concatenated and passed through fully connected layers to yield a binary output: 1 indicates good degradation (defined as $DC_{50} \leq 100\,\text{nM}$ and $D_{max} \geq 80\%$), while 0 indicates poor degradation ($DC_{50} > 100\,\text{nM}$ or $D_{max} < 80\%$). This modular architecture enables DeepPROTACs to effectively model ternary complex formation and predict degradation outcomes based on both structural and sequential representations.

ET-PROTACs. ET-PROTACs is an end-to-end deep learning model designed for PROTAC degradation prediction, consisting of four main components: (i) initial PROTAC and protein featurization; (ii) representation learning using 3D graph-based and sequence-based encoders; (iii) a cross-modal ternary attention block; and (iv) a final classifier. Each component is described in detail below. Each PROTAC is represented as a 2D molecular graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and associated with a 3D coordinate matrix $X \in \mathbb{R}^{|\mathcal{V}|\times 3}$ using RDKit[1]. Nodes $v_i \in \mathcal{V}$ represent atoms, and edges $a_{ij} \in \mathcal{E}$ denote chemical bonds. Node features h_i and edge features a_{ij} are encoded using learned embeddings. Additionally, atomic 3D coordinates $x_i \in \mathbb{R}^3$ are embedded to capture spatial information. The input protein is given as a sequence $P = \{a_1, a_2, \ldots, a_n\}$ of 23 amino acids (including one non-standard residue). Protein sequences are encoded by two embedding layers: a learned character embedding of dimension 64, and a positional embedding of the same dimension. The molecular graph and 3D coordinates are processed by an Equivariant Graph Neural Network (EGNN), which is invariant to rotations and translations. Let h_i^ℓ, x_i^ℓ, and a_{ij} be the node features, coordinates, and edge features at layer ℓ, respectively. The EGNN updates are as follows:

$$\begin{aligned} m_{ij} &= \phi_e(h_i^\ell, h_j^\ell, \|x_i^\ell - x_j^\ell\|^2, a_{ij}), \\ x_i^{\ell+1} &= x_i^\ell + C\sum_{j\neq i}(x_i^\ell - x_j^\ell)\phi_x(m_{ij}), \\ m_i &= \sum_{j\in\mathcal{N}(i)} m_{ij}, \\ h_i^{\ell+1} &= \phi_h(h_i^\ell, m_i), \end{aligned} \tag{1}$$

where $C = 1/(M-1)$, ϕ_e, ϕ_x, and ϕ_h are learnable functions, and M is the number of atoms. Final PROTAC embeddings are obtained by combining features of the warhead, linker, and E3 ligase substructures. ET-PROTACs leverages CNNs to encode protein sequences directly from FASTA format. The sequence embedding E is passed through three CNN layers to yield latent protein features $P_{\text{cnn}} \in \mathbb{R}^{d\times f}$, where f is the number of filters in the last layer. Given feature matrices $D = \{d_1, \ldots, d_X\}$ (PROTAC), $P = \{p_1, \ldots, p_Y\}$ (protein), and $L = \{l_1, \ldots, l_Z\}$ (ligase), we form composite pairs:

$$\begin{aligned} \text{pd}_i &= \text{CONCAT}(p_j, d_i), \\ \text{dl}_j &= \text{CONCAT}(d_i, l_k), \\ \text{pda}_i &= F(W_{\text{pd}} \cdot \text{pd}_i + b), \\ \text{dla}_j &= F(W_{\text{dl}} \cdot \text{dl}_j + b), \end{aligned} \tag{2}$$

where F is a non-linear activation, and $W_{\text{pd}}, W_{\text{dl}} \in \mathbb{R}^{f\times f}$ are learned weights. The combined attention is computed as:

$$\begin{aligned} A_{i,j} &= F(W_a \cdot (\text{pda}_i + \text{dla}_j) + b), \\ A_{\text{pd}} &= \sigma\left(\text{MEAN}(A, 2)\right), \\ A_{\text{dl}} &= \sigma\left(\text{MEAN}(A, 1)\right), \end{aligned} \tag{3}$$

where σ denotes the sigmoid function. The resulting outputs V_{pd} and V_{dl} are the attention-enriched representations of PROTAC–protein and PROTAC–ligase interactions. The attention outputs are pooled over sequence length and concatenated:

$$s = \text{LeakyReLU}(W_s \cdot \text{CONCAT}(V_{\text{pd}}, V_{\text{dl}}) + b_s), \tag{4}$$

where W_s is a learnable weight matrix. Dropout is applied before and after each linear layer to prevent overfitting.

DeepFRI. DeepFRI is a deep learning framework for protein function prediction based solely on amino acid sequences. Given a protein sequence of length L, the input is encoded as a binary matrix $X = [x_1, \ldots, x_L] \in \{0,1\}^{L\times 26}$, where each x_i is a one-hot vector indicating the amino acid type at position i, including 20 standard residues, 5 non-standard residues, and a gap symbol. This input is passed through a set of one-dimensional convolutional layers, each consisting of $f_n = 512$ filters of varying kernel lengths f_l, followed by rectified linear unit (ReLU) activation, $\text{ReLU}(x) = \max(x, 0)$, and a global max-pooling operation. The use of multiple filter sizes enables the extraction of complementary local patterns from the sequence. Outputs from 16 such CNN layers are concatenated, resulting in an $L \times 8192$ feature representation. Finally, a fully connected layer with sigmoid activation outputs probabilities for either Gene Ontology (GO) terms or Enzyme Commission (EC) classes. The output dimensionality is task-specific, corresponding to |GO| or |EC| respectively.

DPFunc. DPFunc is a structure-aware protein function prediction framework that captures residue-level information by integrating 3D geometric structure and pretrained sequence embeddings. For a given protein of length l, a residue-level undirected graph is constructed where each node represents a residue, and an edge is added between two residues if the distance between their C_α atoms is less than 10 Å. This defines the adjacency matrix $A \in \{0,1\}^{l\times l}$. Node features are initialized using embeddings from a pretrained protein language model (ESM-1b), resulting in $X \in \mathbb{R}^{l\times d}$. Two graph convolutional layers are then used to propagate structural information, with residual connections added to facilitate gradient flow. The update rule for the k-th GCN layer is given by:

$$X^{(k+1)} = X^{(k)} + \text{ReLU}\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}X^{(k)}W^{(k)}\right), \tag{1}$$

where $\tilde{A} = A + I$ adds self-loops and $\tilde{D}$ is its corresponding degree matrix. After GCN propagation, domain-level information is introduced to enhance residue representations. Protein domains are annotated using InterProScan, and their one-hot encoding

[1]https://www.rdkit.org/

$IPR \in \{0,1\}^{1\times m}$ is mapped to dense embeddings via two fully connected layers with ReLU activations:

$$H = \text{ReLU}\left((\text{ReLU}(IPR \cdot W_{\text{emd}})W_1 + b_1)\, W_2 + b_2\right). \tag{2}$$

A domain-guided attention mechanism, inspired by the Transformer encoder, is applied to model the interaction between domain and residue representations. For each attention head i, the query, key, and value matrices are computed as:

$$Q_i = HW_i^Q, \quad K_i = X_{\text{final}}W_i^K, \quad V_i = X_{\text{final}}W_i^V, \tag{3}$$

and attention weights are derived as:

$$W_i^A = \text{Softmax}\left(\frac{K_i Q_i^\top}{\sqrt{d}}\right). \tag{4}$$

The multi-head attention output is computed as:

$$X_{\text{multi}} = \text{LayerNorm}\left(\text{Concat}(W_1^A V_1, \ldots, W_n^A V_n)W_{\text{trans}} + X_{\text{final}}\right), \tag{4}$$

followed by a feedforward transformation with residual connection:

$$X_{\text{out}} = \text{LayerNorm}\left(\text{FF}(X_{\text{multi}}) + X_{\text{multi}}\right). \tag{5}$$

Protein-level features are then obtained by summing across residues:

$$x_{\text{pool}} = \sum_{i=1}^{l} X_{\text{out}}[i]. \tag{6}$$

This representation is concatenated with the average of initial residue features:

$$x_{\text{integrate}} = \text{Concat}\left(x_{\text{pool}}, \frac{1}{l}\sum_{i=1}^{l} X[i]\right), \tag{7}$$

and passed through a multilayer perceptron with sigmoid activation to produce GO term probabilities:

$$\hat{y} = \text{Sigmoid}(\text{MLP}(x_{\text{integrate}})). \tag{8}$$

To ensure consistency with the GO hierarchy, a postprocessing step enforces that if a child term is predicted, all its ancestors are also predicted:

$$\hat{y}_i^{\text{post}} = \max\left(\hat{y}_i, \hat{y}_{\text{child}_1}, \ldots, \hat{y}_{\text{child}_n}\right). \tag{9}$$

This hierarchical correction is applied only after inference, without affecting training efficiency.

UniZyme. UniZyme is a biochemically informed framework designed to generalize protein cleavage site prediction across diverse enzymes. It comprises an enzyme encoder and a substrate encoder. The enzyme encoder integrates sequence-derived features with energetic frustration and 3D structural information. Given an enzyme $P_e = (X, R)$, where X are residue embeddings and R are C^α coordinates, the pairwise frustration score is defined as:

$$F(i,j) = \frac{E(i,j) - \mu_{\text{rand}}(i,j)}{\sigma_{\text{rand}}(i,j)}. \tag{1}$$

To incorporate spatial and energetic cues into the self-attention mechanism, each residue pair is encoded via Gaussian basis kernels:

$$\Phi_{i,j}^{\text{energy}} = \text{MLP}(\phi_{\text{energy}}(F(i,j))), \quad \Phi_{i,j}^{\text{dist}} = \text{MLP}(\phi_{\text{dist}}(\|r_i - r_j\|_2)). \tag{2}$$

These terms are added to the attention score matrix in the graph transformer:

$$A_{i,j}^k = \frac{(h_i^{k-1}W_Q)(h_j^{k-1}W_K)^T}{\sqrt{d}} + \Phi_{i,j}^{\text{energy}} + \Phi_{i,j}^{\text{dist}}. \tag{3}$$

To guide the encoder toward catalytically relevant regions, an auxiliary active-site prediction is introduced:

$$\hat{a}_i = \sigma(h_i \cdot w_a). \tag{4}$$

The predicted probabilities $\hat{a}_i$ are then used in a soft attention-like pooling mechanism for enzyme representation:

$$h_e = \sum_{i=1}^{N} \frac{f(\hat{a}_i)}{\sum_j f(\hat{a}_j)} h_i. \tag{5}$$

For a substrate P_s, cleavage site prediction is formulated by concatenating local substrate representations $H_s^{t:t+l}$ with the enzyme representation h_e:

$$\hat{c}_{e,s}^{(t)} = \text{MLP}(\text{CONCAT}(H_s^{t:t+l}, h_e)). \tag{6}$$

The model is jointly optimized using binary cross-entropy losses for both tasks:

$$\mathcal{L} = \mathcal{L}_c(D_c) + \lambda\mathcal{L}_a(D_a). \tag{7}$$

MONN. MONN is composed of four interconnected modules: a graph convolutional module for molecular representation, a CNN module for residue-level protein representation, a pairwise interaction module for atom-residue interaction estimation, and an affinity prediction module for compound-protein binding affinity estimation.

Given a molecular graph $G = (V, E)$, each atom $v_i \in V$ is initially represented by an 82-dimensional one-hot vector v_i^{init} encoding atomic features. These are projected into a hidden space $\mathbb{R}^{h_1}$ by:

$$v_i^0 = f(W_{\text{init}} v_i^{\text{init}}), \tag{1}$$

where $f(x) = \max(0, x) + 0.1\min(0, x)$ is the leaky ReLU activation, and $W_{\text{init}} \in \mathbb{R}^{h_1 \times 82}$. Bonds $e_{i,j} \in E$ are represented by 6-dimensional one-hot vectors encoding bond type and topology.

For L graph convolutional iterations, features are updated by message passing and graph warp. At each layer l, local messages are aggregated:

$$t_i^l = \sum_{v_k \in \mathcal{N}(v_i)} f(W_{\text{gather}}^l [v_k^{l-1}, e_{i,k}]), \tag{2}$$

followed by feature updates:

$$u_i^l = f(W_{\text{update}}^l [t_i^l, v_i^{l-1}]). \tag{3}$$

Global information is captured via a super node s^l that interacts with all u_i^l to yield the final atom features $\{v_i^L\}$ and compound feature s^L.

Protein sequences are encoded by mapping residues to BLOSUM62 columns and processed through a 1D CNN to obtain residue embeddings $\{r_j\}$ in $\mathbb{R}^{h_1}$.

Atom-residue interactions are predicted by projecting atom and residue embeddings to a shared space and computing their dot-product, followed by a sigmoid:

$$P_{i,j} = \sigma(f(W_{\text{atom}} v_i) \cdot f(W_{\text{residue}} r_j)). \tag{4}$$

For affinity prediction, atom, residue, and super node features are transformed via:

$$h_{v,i} = f(W_v v_i), \quad h_s = f(W_s s), \quad h_{r,j} = f(W_r r_j), \tag{5}$$

where $W_v, W_r, W_s \in \mathbb{R}^{h_2 \times h_1}$. A modified dual attention network uses the interaction matrix P to compute attentions $\{\alpha_{v,i}\}, \{\alpha_{r,j}\}$, aggregating compound and protein representations as:

$$h_c = \sum_i \alpha_{v,i} h_{v,i}, \quad h_p = \sum_j \alpha_{r,j} h_{r,j}. \tag{6}$$

The binding affinity is predicted by a regression on the outer product between $[h_c, h_s]$ and h_p:

$$a = W_{\text{affinity}} f(\text{flatten}([h_c, h_s] \otimes h_p)), \tag{7}$$

where $W_{\text{affinity}} \in \mathbb{R}^{1 \times 2h_2^2}$.

CLIPZyme. CLIPZyme formulates enzyme screening as a retrieval task, where a predefined set of enzymes is ranked based on their predicted ability to catalyze a given chemical reaction. Each reaction R and enzyme P is encoded into a d-dimensional vector $r, p \in \mathbb{R}^d$ using learned encoders f_{rxn} and f_{p}, respectively. A scoring function $s(r, p)$ computes the cosine similarity between the two embeddings:

$$s_{ij} = s(r_i, p_j) = \frac{r_i \cdot p_j}{\|r_i\| \|p_j\|}. \tag{1}$$

To align reactions with their catalyzing enzymes, a symmetric contrastive loss is used:

$$\mathcal{L}_{ij} = -\frac{1}{2N} \left(\log \frac{e^{s_{ij}/\tau}}{\sum_k e^{s_{ik}/\tau}} + \log \frac{e^{s_{ij}/\tau}}{\sum_k e^{s_{kj}/\tau}} \right), \tag{2}$$

where τ is a temperature parameter and negative samples are drawn from other enzymes in the batch.

To represent a reaction, atom-mapped molecular graphs of the reactants G_x and products G_y are first encoded by a directed message passing neural network(DMPNN) f_{mol}, yielding atom features a_i and bond features b_{ij}:

$$a_i, b_{ij} = f_{\text{mol}}(G_x, G_y). \tag{3}$$

A pseudo-transition state graph G_{TS} is then constructed using shared atom features and summed bond features:

$$v_i^{(TS)} = v_i^{(x)} = v_i^{(y)}, \tag{3}$$

$$e_{ij}^{(TS)} = b_{ij}^{(x)} + b_{ij}^{(y)}. \tag{4}$$

This graph is processed by a second DMPNN f_{TS}, and the reaction embedding is obtained by aggregating the learned node features:

$$a'_i, b'_{ij} = f_{TS}(G_{TS}), \tag{4}$$

$$r = \sum_i a'_i. \tag{5}$$

Each protein is modeled as a 3D graph $G_p = (V, E)$ with node features h_i, edge features e_{ij}, and 3D coordinates $c_i \in \mathbb{R}^3$. Residue features are initialized using ESM-2 (650M) embeddings with dimensionality 1280. An EGNN with coordinate updates is used to encode G_p into the final protein embedding $p = f_{\text{p}}(G_p)$. Relative distances between residues are encoded using a sinusoidal basis to enhance structural modeling.